Autotest de IP

Banco completo: 170 preguntas con la respuesta correcta marcada (✔) y su explicación.

Tema 0 · Algoritmos, programas y procesos

1. Según la definición del tema 0, un computador es…

A Un conjunto de algoritmos almacenados en memoria

B Un mecanismo digital de propósito general que se convierte en un mecanismo de uso específico cuando procesa un algoritmo ✔

C Un mecanismo analógico diseñado para una única tarea

D Un programa que traduce algoritmos a lenguaje máquina

> El mismo hardware sirve para cualquier tarea: lo que lo «especializa» en cada momento es el algoritmo (el programa) que está ejecutando.

2. Un algoritmo es…

A Una secuencia de instrucciones que especifican las acciones que debe ejecutar un computador para realizar una tarea ✔

B El conjunto de valores de las variables en un instante

C Un programa en ejecución

D Un lenguaje de programación de alto nivel

> El algoritmo describe qué hay que hacer y en qué orden. Un programa en ejecución es un proceso, y los valores de las variables forman el estado.

3. «La noción de algoritmo es más general». En el par partitura / tocar una sonata…

A Ninguno es un algoritmo porque no los ejecuta un computador

B La partitura es el algoritmo y tocar la sonata es el problema (la tarea) ✔

C Los dos son algoritmos

D La partitura es el problema y tocar la sonata es el algoritmo

> Igual que la receta (algoritmo) para cocinar un bizcocho (problema) o el método de Newton para calcular raíces: un algoritmo no tiene por qué ejecutarlo un ordenador.

4. Un proceso es…

A **Un programa en ejecución, caracterizado por una sucesión de estados ✔**

B Un fichero con el código fuente

C La expresión de un algoritmo en un lenguaje de programación

D Una instrucción máquina

El programa es el texto; el proceso es lo que ocurre al ejecutarlo: va pasando de un estado inicial a un estado final.

5. El estado de un proceso viene dado por…

A Sólo los valores de las variables

B El código fuente y el compilador usado

C Sólo la instrucción que se está ejecutando

D **Los valores de las variables y un indicador de la siguiente instrucción a ejecutar ✔**

El estado determina el grado de progreso del proceso: qué valen las variables y por dónde va.

6. Si ejecutas dos veces el mismo programa con datos distintos (5 y 4), obtienes…

A El mismo proceso dos veces

B Dos algoritmos distintos

C **Dos procesos distintos del mismo programa ✔**

D Dos programas distintos

El programa es el mismo; cada ejecución es un proceso con su propia sucesión de estados.

7. Una variable es…

A Una instrucción de lectura

B Un tipo de dato

C Un valor constante del programa

D **Una magnitud o información manejada por un algoritmo a la que se da un nombre y que registra un valor ✔**

Representa una entidad del mundo real relevante en el problema (por ejemplo, la altura) y su valor puede cambiar durante el proceso.

8. Un problema es computable cuando…

A Lo puede resolver un ordenador en menos de un segundo

B Existe un algoritmo de orden polinomial que lo resuelve

C Se puede expresar en C++

D **Existe un algoritmo que lo resuelve ✔**

Computable = existe algoritmo. Si además existe un algoritmo eficiente (orden polinomial), es tratable.

9. Un problema computable es tratable si…

A Existe al menos un algoritmo que lo resuelve

B No existe ningún algoritmo que lo resuelva

C Se resuelve con una sola instrucción máquina

D Existe un algoritmo eficiente (de orden polinomial) que lo resuelve ✔

> Los computables se dividen en tratables (algoritmo polinomial) e intratables. Además hay problemas no computables.

10. ¿Cuál es la clasificación de problemas del tema 0?

A Tratables y computables

B Secuenciales, alternativos e iterativos

C Computables (tratables e intratables) y no computables ✔

D Numéricos y no numéricos

> Problemas → computables → {tratables, intratables}; y no computables.

11. El formato de una instrucción máquina es…

A Siempre un código y tres operandos

B Un algoritmo completo

C Una línea de C++

D Código de operación + entre 0 y 3 operandos ✔

> El código indica la operación (suma, resta, comparación, salto…) y los operandos pueden ser direcciones de memoria o valores.

12. ¿Por qué hacen falta compiladores e intérpretes?

A Porque el computador sólo ejecuta instrucciones máquina muy simples y hay que traducir los programas escritos en lenguajes de más alto nivel ✔

B Porque los algoritmos no se pueden escribir en papel

C Porque el ordenador no tiene memoria suficiente

D Para convertir los procesos en programas

> Los lenguajes de programación expresan las operaciones con más abstracción que el lenguaje máquina; un traductor las convierte en instrucciones máquina.

13. ¿Por qué se suele usar una notación algorítmica al analizar un problema?

A Para evitar los detalles propios de un lenguaje concreto y centrarse en el proceso que debe hacer el computador ✔

B Porque es más rápida de ejecutar

C Porque el compilador la entiende directamente

D Porque los lenguajes de programación no permiten variables

> La notación (nuestra «SP») se centra en el qué; los detalles del lenguaje (cin, cout, tipos concretos…) se dejan para la codificación.

14. Sobre algoritmos y lenguajes de programación es cierto que…

- A Los lenguajes de programación son instrucciones máquina
- B Cada algoritmo sólo se puede escribir en un lenguaje
- C Un algoritmo es lo mismo que un programa en C++
- **D El mismo algoritmo se puede expresar en lenguajes de programación diferentes ✔**

> El programa es la expresión de un algoritmo en un lenguaje concreto; el mismo algoritmo puede dar lugar a programas en C, Python, C++…

15. La diferencia entre algoritmo y programa es que…

- A No hay diferencia
- B El programa no tiene variables
- **C El programa es un algoritmo escrito en un lenguaje de programación concreto ✔**
- D El algoritmo se ejecuta y el programa no

> El algoritmo del tema 0 (Léxico numero: entero… Leer, Escribir) y su programa en C++ hacen lo mismo; cambia la notación.

Tema 1 · Léxico y organización de un algoritmo

16. Un algoritmo está formado por…

- **A El léxico (informaciones y acciones) y el control ✔**
- B Sólo instrucciones de lectura y escritura
- C Sólo variables
- D Un compilador y un programa

> Construir un algoritmo es elegir informaciones y acciones (léxico) y decidir cómo organizarlas en el tiempo (control).

17. Las tres formas de control (organización de las acciones) son…

- A Lectura, escritura y asignación
- B Variables, constantes y tipos
- C Precondición, postcondición e invariante
- **D Secuenciación, análisis de casos e iteración (o recursión) ✔**

> El control determina el orden y las condiciones de ejecución de las acciones.

18. La abstracción consiste en…

- A Escribir el algoritmo directamente en C++
- B Evitar usar variables
- **C Eliminar detalles innecesarios y considerar lo esencial ✔**
- D Añadir todos los detalles posibles al léxico

Es el mecanismo fundamental para dominar la complejidad. El léxico (variables y acciones) fija el nivel de abstracción.

19. En el léxico, de una acción se especifica…

- A Sólo su nombre
- B Su valor inicial
- C Su nombre y su tipo
- **D Su nombre, su precondición y su postcondición ✔**

De una información (variable) se dan nombre y tipo; de una acción, nombre, precondición y postcondición.

20. La precondición de una acción es…

- **A La condición que debe cumplirse antes de ejecutarla ✔**
- B La primera instrucción de su algoritmo
- C El tipo de sus parámetros
- D La condición que se cumple al terminarla

Ejemplo: para retirar, la cantidad debe ser mayor que 0 y no superar el saldo.

21. La postcondición de una acción es…

- **A La condición que debe cumplirse como resultado de su ejecución ✔**
- B Un comentario opcional sin significado
- C La condición que debe cumplirse antes de ejecutarla
- D Su última instrucción

Ejemplo: tras retirar, el nuevo saldo es el anterior menos la cantidad retirada.

22. Un tipo de datos especifica…

- A La memoria que ocupa un programa
- B El número de variables del algoritmo
- **C Un dominio de valores y el conjunto de operaciones aplicables a ese dominio ✔**
- D Sólo el nombre de una variable

Para cada tipo hay que conocer: dominio, operaciones, sintaxis de los literales y sintaxis de las expresiones.

23. En la notación, `17 DIV 5` y `17 MOD 5` valen…

A **3 y 2 ✔**

B 2 y 3

C 3,4 y 2

D 3 y 0

> DIV es el cociente entero y MOD el resto: 17 = 5·3 + 2.

24. En la notación, el operador `/` aplicado a dos enteros…

A Da el resto

B **Da un Real: `7 / 2` = 3.5 ✔**

C No está definido

D Da un Entero: `7 / 2` = 3

> En la tabla de operaciones del tema 1, `/ : Entero × Entero → Real`. Para el cociente entero se usa DIV. (¡En C++ es distinto!)

25. Con `númeroAlumnos = 200` y `cursos = 4`, ¿cuánto vale `1000 - númeroAlumnos * 2`?

A **600 ✔**

B 1600

C −400

D Error de tipos

> La multiplicación tiene prioridad: 1000 − 400 = 600 (Entero).

26. Con `nota = 6.0` y `peso = 80.0`, ¿cuánto vale `nota * peso / 100.0 - 3.0`?

A 4.8

B 1.2

C **1.8 ✔**

D −13.2

> 6·80 = 480; 480/100 = 4.8; 4.8 − 3 = 1.8. Con paréntesis, `nota * (peso/100.0 - 3.0)` = 6 · (−2.2) = −13.2.

27. Con `letra = 'A'` y `aprobado = Verdadero`, ¿cuánto vale `NO (letra = 'B') O aprobado`?

A 'B'

B No está definido

C Falso

D **Verdadero ✔**

> letra = 'B' es Falso, NO Falso = Verdadero, y Verdadero O lo que sea = Verdadero.

28. Con `nota = 6.0` y `media = 5.0`, ¿cuánto vale `(nota > 7.0) Y (media = 5.0)`?

A **Falso ✔**

B Verdadero

C 5.0

D 6.0

nota > 7.0 es Falso y con Y basta un Falso para que el resultado sea Falso. El tipo del resultado es Booleano.

29. `Carácter (66)` y `Sucesor ('A')` valen…

A 66 y 'B'

B 'A' y 'C'

C 'B' y 66

D **'B' y 'B' ✔**

Carácter (o Car) pasa de código a carácter (66 → 'B' en ASCII) y Sucesor da el siguiente carácter del código.

30. ¿Cuál es el tipo de la operación `Ord`?

A Carácter → Carácter

B Entero → Carácter

C **Carácter → Entero ✔**

D Entero → Booleano

Ord da el código de un carácter; Car hace lo contrario (Entero → Carácter).

31. La semántica de la asignación `v ← expr` es…

A Leer v del teclado

B Comparar v con expr

C **Evaluar la expresión de la derecha y asignar el resultado a la variable de la izquierda ✔**

D Asignar v a la expresión

Es la acción primitiva que caracteriza a los lenguajes imperativos. Sólo puede haber una variable a la izquierda.

32. Tras `longitud ← 4.7; longitud ← longitud / 2`, la variable longitud vale…

A 2

B 4.7

C **2.35 ✔**

D 9.4

El nuevo valor de longitud es función del anterior: 4.7 / 2 = 2.35.

33. La acción `Leer (radio)`…

A Declara la variable radio

B Comprueba que radio sea positivo

C Asigna a radio el valor introducido por teclado ✔

D Muestra radio por pantalla

> Leer recibe datos del terminal (teclado) y los asigna a las variables de la lista; Escribir envía al terminal los valores de una lista de expresiones.

34. La composición secuencial reduce la complejidad…

A Introduciendo estados intermedios: el problema se descompone en subproblemas más simples que se resuelven por separado ✔

B Eligiendo una acción según los datos

C Repitiendo una acción hasta que se cumple una condición

D Eliminando variables

> {E0} A1 {E1} A2 {E2} … {En}: cada acción lleva de un estado al siguiente.

35. Aplicando el algoritmo del tema 1, 309639 segundos equivalen a…

A 3 días, 14 horas, 39 minutos y 0 segundos

B 3 días, 14 horas, 0 minutos y 39 segundos ✔

C 3 días, 13 horas, 60 minutos y 39 segundos

D 85 horas, 0 minutos y 39 segundos

> d = 309639 DIV 86400 = 3, rd = 50439; h = 50439 DIV 3600 = 14, rh = 39; m = 39 DIV 60 = 0, s = 39.

36. ¿Cuántos días, horas, minutos y segundos son 100000 segundos?

A 1 d, 4 h, 6 min, 40 s

B 27 h, 46 min, 40 s y 0 d

C 1 d, 3 h, 46 min, 40 s ✔

D 1 d, 3 h, 40 min, 46 s

> 100000 − 86400 = 13600; 13600 DIV 3600 = 3, resto 2800; 2800 DIV 60 = 46, resto 40.

37. Para que una descomposición por análisis de casos sea correcta debe…

A Incluir siempre EN_OTRO_CASO

B Cubrir todo el dominio inicial y que los subdominios no se solapen ✔

C Estar guiada por los resultados

D Tener exactamente dos casos

> Además, la postcondición de cada subproblema debe cumplir la del problema inicial.

38. ¿Qué falla en esta clasificación por edad en [0, 120]? `0 ≤ edad ≤ 12`: infantil; `12 ≤ edad ≤ 18`: joven; `edad > 20`: adulto.

- A Es correcta
- B Sólo se solapan en 18
- **C Se solapan en 12 y no se cubren las edades entre 18 y 20 (19 y 20) ✔**
- D Sólo falta cubrir el 0

> Son los dos errores típicos: olvidar casos (19, 20) e incluir casos repetidos (12 está en dos subdominios).

39. Calcular el porcentaje de impuesto según la renta (renta baja, media, alta) es una descomposición por casos guiada por…

- A Los parámetros
- **B Los datos de entrada ✔**
- C La iteración
- D Los resultados

> Se particiona el dominio de la entrada (ingresos). Determinar el estado académico (sobresaliente, notable…) a partir del rango del resultado es el ejemplo guiado por resultados.

40. En `SEGÚN … e1: a1 … en: an EN_OTRO_CASO: an+1 FIN_SEGÚN`, la acción an+1 se ejecuta si…

- A La última condición es verdadera
- **B Todas las condiciones e1 … en son falsas ✔**
- C Alguna condición es falsa
- D Siempre, al final

> EN_OTRO_CASO recoge lo que no cubre ninguna de las condiciones: e1 O e2 O … O en = Falso.

41. `SI e ENTONCES a FIN_SI` equivale a…

- A `SEGÚN … e: a no(e): a FIN_SEGÚN`
- **B `SEGÚN … no(e): ; (acción vacía) e: a FIN_SEGÚN` ✔**
- C `SEGÚN … e: a FIN_SEGÚN` sin más casos, que deja casos sin cubrir
- D `MIENTRAS e HACER a FIN_MIENTRAS`

> El SI sin SI_NO también cubre todo el dominio: si e es falsa se ejecuta la acción vacía.

42. Para ordenar tres enteros distintos con un único SEGÚN, ¿cuántos casos hacen falta?

- A 8
- **B 6 ✔**
- C 3
- D 9

> El resultado es una permutación de X, Y, Z: hay 3! = 6 órdenes posibles (a<b<c, a<c<b, b<a<c, b<c<a, c<a<b, c<b<a).

43. Postcondición correcta para `z = máximo de X e Y`:

A `(z = X o z = Y) y z ≥ X y z ≥ Y` ✔

B `z = X o z = Y`

C `z > X y z > Y`

D `z ≥ X y z ≥ Y`

Hacen falta las dos partes: z es uno de los dos valores y es mayor o igual que ambos.

44. El dominio de un producto de tipos (registro) `TIPO < a1: T1; …; aN: TN >` es…

A La unión de los dominios de T1 … TN

B Una secuencia de longitud variable

C Las N-tuplas de valores de los tipos constituyentes ✔

D Sólo el dominio de T1

Un valor es una agregación de un valor de cada campo, por ejemplo <“J. Gómez”, 21, F, 4, 3>.

45. ¿Cuántos valores distintos tiene el tipo `TIPO < x: Booleano; y: Entero [1, 6] >`?

A 6

B 8

C 2

D 12 ✔

La cardinalidad de un producto es el producto de cardinalidades: 2 · 6 = 12.

46. Para acceder al campo curso de la variable a1 de tipo Estudiante se escribe…

A `Estudiante.curso`

B `a1[curso]`

C `a1.curso` (o «curso de a1») ✔

D `curso.a1`

El operador punto (o «de») selecciona un campo de una variable de tipo registro.

47. En la notación, `númeroAlumnos / cursos` con 200 y 4 es…

A Un error

B Un Booleano

C Un Entero: 50

D Un Real: 50.0 ✔

`/` entre enteros da Real; para un Entero se usa DIV (`númeroAlumnos DIV cursos` = 50).

Tema 2 · Acciones, funciones y tablas

48. Una acción (procedimiento)…

A Calcula un valor que se usa dentro de expresiones

B Sólo puede tener parámetros dato

C Es un tipo de dato estructurado

D Describe una acción compleja en función de otras más simples o primitivas y se usa como una acción primitiva ✔

> Su algoritmo combina acciones más simples para modificar el estado y lograr la postcondición a partir de la precondición.

49. ¿Por qué conviene parametrizar la acción AñoBisiesto con `dato f: Fecha`?

A Para que devuelva un Booleano

B Para poder usarla con cualquier fecha (hoy, mañana…): más independencia del contexto y más reutilizable ✔

C Para que se ejecute más rápido

D Porque las acciones no pueden usar variables globales

> La versión sin parámetro sólo servía para la variable f del léxico global. «Mayor parametrización supone mayor independencia del contexto».

50. ¿Cómo se corresponden las clases de parámetros con el paso en C++?

A dato → por valor; resultado y dato–resultado → por referencia (&) ✔

B Todos por referencia

C Todos por valor

D dato → por referencia; resultado → por valor

> Los parámetros dato sólo entran; los resultado y dato–resultado tienen que devolver valores al argumento, así que se pasan por referencia.

51. Un parámetro dato–resultado es aquel cuyo valor…

A No se usa

B Inicial es necesario para la acción y además es modificado por ella ✔

C Sólo se suministra a la acción y no se modifica

D Sólo se calcula dentro de la acción

> Ejemplo: los parámetros x, y de Intercambio: se usan sus valores y se modifican.

52. Si un parámetro es DATO, el argumento de la llamada puede ser…

A Sólo otra acción

B Sólo un literal

C Cualquier expresión del mismo tipo ✔

D Sólo una variable

> En la llamada: parámetro ← argumento. Si es DATO–RESULTADO o RESULTADO, el argumento debe ser una variable, porque al volver se hace argumento ← parámetro.

53. Al emparejar los parámetros formales con los argumentos deben coincidir…

A El número, el tipo y la posición ✔

B Sólo el tipo

C Los identificadores

D Sólo el número

> Los identificadores no tienen por qué coincidir: los parámetros son parte del léxico local de la acción.

54. Con `a = 7, b = 4`, tras `Intercambio (a, b)` (parámetros dato–resultado)…

A a = 7 y b = 7

B a = 4 y b = 7 ✔

C a = 7 y b = 4

D a = 4 y b = 4

> Al entrar, x = 7, y = 4; t ← x; x ← y; y ← t deja x = 4, y = 7, y al volver se copian a a y b.

55. Si los parámetros de Intercambio fueran DATO (paso por valor), tras `Intercambio (a, b)` con a = 7, b = 4…

A Daría un error de compilación

B a = 4 y b = 7

C a y b valdrían 4

D a y b seguirían valiendo 7 y 4 ✔

> Se intercambiarían las copias locales x e y, pero nada vuelve a los argumentos.

56. Con a = 12, b = 80, c = 25, ¿qué queda tras `Intercambio (a, b); Intercambio (b, c)`?

A a = 80, b = 12, c = 25

B a = 80, b = 25, c = 12 ✔

C a = 25, b = 12, c = 80

D a = 12, b = 80, c = 25

> (12, 80, 25) → (80, 12, 25) → (80, 25, 12): desplazamiento circular con un estado intermedio.

57. Con `ValDec_NumBin (dato bit3, bit2, bit1, bit0; resultado valDec)` que calcula `bit3*8 + bit2*4 + bit1*2 + bit0`, ¿qué devuelve `ValDec_NumBin (1, 1, 1, 0, vD)`?

A 7

B 1110

C 14 ✔

D 13

> 8 + 4 + 2 + 0 = 14.

58. Con x = 0 e y = 1, ¿qué devuelve `ValDec_NumBin (y, x, y, x, vD)` (bit3·8 + bit2·4 + bit1·2 + bit0)?

A 5

B 10 ✔

C 9

D 2

> bit3 = 1, bit2 = 0, bit1 = 1, bit0 = 0 → 1010 en binario = 8 + 2 = 10.

59. La abstracción por especificación separa…

A Los datos de los resultados

B Qué hace una acción (especificación) de cómo lo hace (implementación) ✔

C El léxico global del local

D Las acciones de las funciones

> Para usar Intercambio basta saber {x = X, y = Y} → {x = Y, y = X}; no hace falta mirar su algoritmo. La abstracción por parametrización da generalidad (vale para dos variables cualesquiera).

60. «Localidad» de las acciones parametrizadas significa que…

A Sólo pueden usar variables locales

B Sólo se pueden llamar una vez

C Se pueden escribir y entender independientemente del contexto en que se usan; para usarlas basta su especificación ✔

D Se ejecutan en la máquina local

> Las acciones son la base de la descomposición de programas en el paradigma imperativo.

61. Una función, a diferencia de una acción…

A No se describe con un algoritmo

B Puede tener parámetros resultado

C Modifica el estado del proceso

D Establece una relación entre dominio y codominio y se usa dentro de expresiones, como un operador ✔

> Las funciones extienden el repertorio de operadores: `t ← a + b + c - Max3 (a, b, c)`.

62. Los parámetros de una función son…

A De cualquier clase

B Todos de tipo DATO ✔

C Al menos uno RESULTADO

D Todos DATO–RESULTADO

> La función produce un único resultado, que se asigna a su nombre como último paso; no devuelve valores por parámetros.

63. Dentro de una función, su identificador se considera…

A Un parámetro dato

B Una variable de lectura y escritura

C Una constante

D Una variable de sólo escritura a la que se asigna el resultado como último paso ✔

> Por ejemplo `Max3 ← may` al final del algoritmo de Max3.

64. `Max2 ← ((x+y) + abs (x-y)) DIV 2`. ¿Cuánto vale Max2 (8, 3)?

A 3

B 5

C 11

D 8 ✔

> ((8+3) + 5) DIV 2 = 16 DIV 2 = 8. La fórmula funciona porque x + y + |x − y| = 2·máx(x, y).

65. Con la misma fórmula, ¿cuánto vale Max2 (−2, 5)?

A −2

B 3

C 7

D 5 ✔

> ((−2+5) + 7) DIV 2 = 10 DIV 2 = 5.

66. Max3 por composición funcional se escribe…

A `Max3 ← Max2 (x, y, z)`

B `Max2 (x, y); Max2 (y, z)`

C `Max3 ← Max2 (x, y) + Max2 (y, z)`

D `Max3 ← Max2 (x, Max2 (y, z))` ✔

> El resultado de una función puede ser argumento de otra (incluso de ella misma): Max2 (6, Max2 (3, 5)) = Max2 (6, 5) = 6.

67. Con `Cvds (d) = 3600*d.h + 60*d.m + d.s`, ¿cuánto vale Cvds (<1, 2, 3>)?

A 123

B 62

C 3605

D 3723 ✔

> 3600 + 120 + 3 = 3723 segundos.

68. Cvsd pasa segundos a duración. ¿Cuánto vale Cvsd (3725)?

A <1, 2, 5> ✔

B <1, 2, 50>

C <1, 0, 125>

D <0, 62, 5>

> h = 3725 DIV 3600 = 1, rh = 125; m = 125 DIV 60 = 2; s = 125 MOD 60 = 5.

69. SumarD (<3, 30, 59>, <1, 59, 5>) = Cvsd (Cvds (a) + Cvds (b)) vale…

A <4, 89, 64>

B <5, 30, 4> ✔

C <5, 29, 64>

D <4, 30, 4>

> 12659 + 7145 = 19804 s → 5 h (18000), resto 1804 → 30 min (1800), 4 s.

70. Regla de la máxima localidad:

A Los identificadores de léxicos interiores enmascaran a los idénticos de niveles superiores ✔

B Los parámetros son visibles fuera de la acción

C Las variables globales tienen prioridad sobre las locales

D Una acción no puede tener léxico local

> Si una acción declara una variable t y también hay una t global, dentro de la acción t es la local.

71. Si la acción A contiene en su léxico las acciones A1 y A2, el léxico de A1 tiene ámbito en…

A Todo el algoritmo

B A, A1 y A2

C Sólo A1 ✔

D A1 y A2

> El léxico del algoritmo principal se ve en todas partes; el de A, en A, A1 y A2; el de A1, sólo dentro de A1.

72. ¿Cuántos elementos tiene `Tabla [3, 10] de Real`?

A 10

B 8 ✔

C 7

D 13

> n = ordinal (bs) − ordinal (bi) + 1 = 10 − 3 + 1 = 8.

73. En una tabla, el dominio y el rango son…

A Dominio: reales; rango: enteros

B Dominio: el tipo base; rango: los índices

C Dominio: un intervalo de un tipo ordinal (los índices); rango: el tipo base (cualquiera) ✔

D Dominio y rango son siempre enteros

> `Temperatura = TIPO Tabla [1, 12] de Real`: índices 1…12, elementos reales.

74. Con `T = [10, 9.5, 11, 14.6, …]` (Tabla [1, 12]), ¿cuánto vale T2?

A 10

B 14.6

C 11

D 9.5 ✔

> Acceso directo por índice: T1 = 10, T2 = 9.5. La media de enero y febrero sería (T1 + T2) / 2.

75. Día del año = `30 * (m - 1) + d + correc[m]` con `correc = [0, 1, -1, 0, 0, 1, 1, 2, 3, 3, 4, 4]`. ¿Qué día del año es el 1 de marzo?

A 61

B 60 ✔

C 31

D 59

> 30 · 2 + 1 + correc[3] = 60 + 1 − 1 = 60 (31 de enero + 28 de febrero + 1).

76. ¿Qué distingue una tabla de un producto de tipos?

A Sus elementos son de idéntica naturaleza y se referencian por indexación; el nº de elementos significativos no es fijo ✔

B Una tabla tiene campos con nombre

C Un producto de tipos se recorre con un índice

D No hay diferencias

> En el registro cada campo puede ser de un tipo distinto y se accede por nombre (a1.curso).

77. Las operaciones más frecuentes sobre tablas son…

- A La suma y la resta de tablas
- B Leer y escribir la tabla completa de golpe
- C Intercalar y copiar
- **D El recorrido (misma acción sobre todos los elementos) y la búsqueda (encontrar uno que cumpla una propiedad) ✔**

> Son los mismos esquemas que se estudian para secuencias en el tema 4.

Tema 3 · La iteración y sus formas

78. Cualquier iteración se compone de…

- A Lectura, cálculo y escritura
- B Léxico, algoritmo y fin
- C Condición y nada más
- **D Inicialización, cuerpo y tratamientos de finalización ✔**

> Y siempre debe tener un punto de parada.

79. La composición MIENTRAS…

- A Se ejecuta un número fijo de veces
- B Comprueba la condición al final; el cuerpo se ejecuta al menos una vez
- **C Comprueba la condición al principio; el cuerpo puede no ejecutarse ninguna vez ✔**
- D Para en medio del cuerpo

> La condición del MIENTRAS es de continuación: se repite mientras sea verdadera.

80. La composición REPETIR … HASTA cond…

- A Comprueba al principio y cond es de continuación
- B Puede no ejecutarse nunca
- C Es igual que RECORRIENDO
- **D Comprueba al final, ejecuta el cuerpo al menos una vez y cond es de parada ✔**

> Por eso es la adecuada para pedir un dato hasta que sea válido: hay que pedirlo al menos una vez.

81. En `ITERAR S2 DETENER cond S3 FIN_ITERAR`…

A **S2 se ejecuta al menos una vez y S3 puede no ejecutarse ✔**

B S3 se ejecuta siempre antes que S2

C cond es de continuación

D S2 y S3 se ejecutan al menos una vez

> Es una iteración con parada en el medio; cond es de parada, como en el REPETIR.

82. ¿Qué composición es la natural para pedir una nota hasta que esté entre 0 y 10?

A MIENTRAS con la nota sin leer

B **REPETIR … HASTA QUE nota válida ✔**

C RECORRIENDO [0, 10]

D Ninguna: basta un SI

> Hay que pedir el dato antes de poder comprobarlo: el cuerpo se ejecuta al menos una vez.

83. `MIENTRAS c HACER S2 FIN_MIENTRAS` equivale a…

A `ITERAR S2 DETENER c FIN_ITERAR`

B `REPETIR S2 HASTA c`

C `SI c ENTONCES REPETIR S2 HASTA NO c FIN_SI` ✔

D `REPETIR S2 HASTA NO c`

> Hace falta el SI porque el MIENTRAS puede no ejecutar el cuerpo; y la condición de parada es la negación de la de continuación.

84. `REPETIR S2 HASTA c` equivale a…

A `SI c ENTONCES S2 FIN_SI`

B `MIENTRAS c HACER S2 FIN_MIENTRAS`

C `S2; MIENTRAS NO c HACER S2 FIN_MIENTRAS` ✔

D `MIENTRAS NO c HACER S2 FIN_MIENTRAS`

> Se ejecuta S2 una vez fuera y luego se repite mientras no se cumpla la condición de parada.

85. `ITERAR S2 DETENER c S3 FIN_ITERAR` equivale a…

A `MIENTRAS NO c HACER S2; S3 FIN_MIENTRAS`

B `S2; MIENTRAS NO c HACER S3; S2 FIN_MIENTRAS` ✔

C `REPETIR S2; S3 HASTA c`

D `S2; S3; MIENTRAS c HACER S2 FIN_MIENTRAS`

> Es la equivalencia que se usa en C++, donde no hay ITERAR: `S2; while (!(c)) { S3; S2; }`.

86. ¿Cuántas veces se ejecuta el cuerpo de `i RECORRIENDO [5, 3] HACER …`?

- **A** Da error
- **B** Una
- **C** **Ninguna ✔**
- **D** 3

> Si el límite inferior está por encima del superior el intervalo es vacío y el bloque no se ejecuta.

87. Tras terminar un RECORRIENDO, la variable de control…

- **A** **Tiene un valor no definido (como si no estuviera inicializada) ✔**
- **B** Vale el límite superior
- **C** Vale el límite superior + 1
- **D** Vale el límite inferior

> No se debe usar su valor después del recorrido sin volver a asignarla.

88. Sobre RECORRIENDO es cierto que…

- **A** Siempre se ejecuta al menos una vez
- **B** El cuerpo puede cambiar la variable de control para saltar valores
- **C** **El cuerpo no debe modificar la variable de control y no hay que suponer que los límites se reevalúan ✔**
- **D** Los límites se recalculan en cada vuelta

> Es sano que los límites a y b no cambien durante la ejecución.

89. `p ← 0; i RECORRIENDO [1, b] HACER p ← p + a FIN_RECORRIENDO` calcula…

- **A** **a · b ✔**
- **B** a elevado a b
- **C** b · (b + 1) / 2
- **D** a + b

> Recorre los múltiplos de a: 0, a, 2a, …, b·a. Se usa RECORRIENDO porque el número de vueltas (b) se conoce antes de empezar.

90. `fact ← 1; i RECORRIENDO [2, n] HACER fact ← fact * i FIN_RECORRIENDO` con n = 0 da…

- **A** Error
- **B** 0
- **C** **1, porque el intervalo [2, 0] es vacío ✔**
- **D** 2

> 0! = 1 por definición, y el intervalo vacío lo respeta sin tratarlo aparte.

91. ¿Por qué la versión del factorial con RECORRIENDO parece la más apropiada?

A Porque hace menos multiplicaciones que ninguna otra

B Porque el número de iteraciones se conoce de antemano (n − 1) y la composición lo expresa directamente ✔

C Porque no necesita variables

D Porque es la única que funciona con n = 0

> Con MIENTRAS hay que gestionar a mano el contador; RECORRIENDO deja implícitos el inicio y la parada.

92. Los divisores propios de 12 (recorriendo [2, a − 1]) son…

A 2, 3, 4 y 6 ✔

B 2, 3 y 4

C 2, 4 y 6

D 1, 2, 3, 4, 6 y 12

> Se excluyen 1 y el propio número. Mejora: basta recorrer [2, a DIV 2], porque ningún divisor propio supera a/2.

93. Para decidir si n es primo, el algoritmo del tema busca un divisor desde 2 con `MIENTRAS n MOD j ≠ 0 HACER j ← j + 1`. n es primo si…

A j = 2

B n MOD j = 0 al principio

C j > n

D El primer divisor encontrado es el propio n (j = n) ✔

> Es una búsqueda con garantía de éxito: n siempre se divide a sí mismo, así que el bucle termina.

94. ¿Qué dos mejoras se piden para el algoritmo del primo?

A Usar REPETIR en vez de MIENTRAS

B Descartar los pares (salvo el 2) y buscar divisores sólo hasta la raíz cuadrada ✔

C Empezar a buscar en n y bajar

D Probar sólo con divisores primos menores que 10

> Si a no tiene divisores ≤ √a tampoco los tiene por encima; y basta probar divisores impares.

95. ¿Qué ventaja tiene la segunda versión de «potencias de 2 en [x, y]» frente a la que llama a P2 (i) en cada vuelta?

A Usa menos variables

B Es más corta pero hace más multiplicaciones

C Reutiliza la potencia anterior (multiplica por 2) en lugar de recalcularla desde cero en cada vuelta ✔

D Funciona para x negativo

> La primera hace 1 + 2 + … multiplicaciones (P2 recorre [1, i] cada vez); la segunda, una por término tras calcular 2^x.

96. Con el algoritmo de la raíz entera por suma de impares, ¿qué devuelve para n = 30?

A **5 ✔**

B 6

C 15

D 4

Cuadrados 1, 4, 9, 16, 25, 36: el último ≤ 30 es $25 = 5^2$.

97. La suma de los k primeros números impares (1 + 3 + 5 + …) es…

A **k^2 ✔**

B 2^k

C k(k + 1)/2

D 2k

1 = 1, 1 + 3 = 4, 1 + 3 + 5 = 9… Es la propiedad que usa el algoritmo de la raíz entera.

98. Con f0 = f1 = 1 y fi = fi−1 + fi−2, ¿cuánto vale f5?

A 13

B 3

C **8 ✔**

D 5

1, 1, 2, 3, 5, 8: f5 = 8. (Los diez primeros: 1, 1, 2, 3, 5, 8, 13, 21, 34, 55.)

99. En el cuerpo del Fibonacci iterativo, ¿en qué orden se actualizan las variables?

A `penult ← ult; ult ← fibo; fibo ← ult + penult` **✔**

B El orden da igual

C `fibo ← ult + penult; penult ← ult; ult ← fibo`

D `ult ← fibo; penult ← ult; fibo ← ult + penult`

Primero se «desplazan» los términos y después se calcula el nuevo; cambiar el orden pierde un valor que todavía hace falta.

100. Una iteración siempre debe tener…

A Exactamente una variable

B Un contador

C Un REPETIR

D **Un punto de parada ✔**

Sin punto de parada el proceso no termina.

101. En el cálculo de la media de notas con ITERAR, ¿qué va justo después de `DETENER nota < 0`?

A `Leer (nota)`

B Nada: el ITERAR termina ahí

C `suma ← suma + nota; numElem ← numElem + 1` ✔

D `Escribir (suma / numElem)`

`ITERAR Leer (nota); DETENER nota < 0; suma ← suma + nota; numElem ← numElem + 1 FIN_ITERAR`: la lectura va antes de la parada y el tratamiento después.

102. En la media de notas con MIENTRAS, ¿por qué se comprueba `numElem > 0` al final?

A Porque MIENTRAS siempre ejecuta el cuerpo una vez

B Porque numElem puede ser negativo

C Porque si la primera nota ya es negativa no hay notas y dividir entre 0 no tiene sentido ✔

D No hace falta

Es el tratamiento de finalización: la secuencia vacía necesita un mensaje propio.

103. Para dividir x entre y por restas sucesivas, los valores iniciales y la condición del bucle son…

A c ← 1; r ← 0; MIENTRAS r < y

B c ← x; r ← y; MIENTRAS c ≥ r

C c ← 0; r ← x; MIENTRAS r ≥ y ✔

D c ← 0; r ← y; MIENTRAS r > 0

Se mantiene x = y·c + r y se resta y mientras quepa; al salir 0 ≤ r < y.

104. La composición ITERAR en C++…

A Se escribe `iterate`

B Es el `switch`

C No existe: se expresa con `while` / `do-while` (por ejemplo, `do { S1; if (cond) break; S2; } while (true);`) ✔

D Es el `for`

Es el único caso en el que el boletín admite `break`.

105. ¿Cuántas veces se ejecuta el cuerpo de `i ← 0; MIENTRAS i < n HACER i ← i + 1 FIN_MIENTRAS` con n = 5?

A 6

B 5 ✔

C 0

D 4

i toma 0, 1, 2, 3, 4 al entrar en el cuerpo: 5 vueltas; al llegar a 5 la condición es falsa.

106. `k RECORRIENDO [1, 4] EN SENTIDO INVERSO HACER Escribir (k)` escribe…

A 4

B Nada

C 4 3 2 1 ✔

D 1 2 3 4

> La variante inversa recorre el intervalo de b hasta a.

Tema 4 · Secuencias y esquemas

107. Una secuencia es…

A Un conjunto sin orden de valores del mismo tipo

B Un conjunto de valores de cualquier tipo en el que es significativo el orden; su longitud no se conoce a priori ✔

C Un registro con campos

D Una tabla con índices

> [32, 64, 85] ≠ [85, 64, 32], y [ ] es la secuencia vacía (longitud 0).

108. long (['h', 'o', 'l', 'a']) y long ([ ]) valen…

A 4 y 0 ✔

B 3 y 0

C 5 y 0

D 4 y 1

> La longitud es el número de elementos. La marca de fin no cuenta.

109. En el primer modelo de acceso secuencial, `Comenzar (S)`…

A Avanza al segundo elemento

B Graba la marca de fin

C Inicia la consulta y deja el primer elemento como elemento actual ✔

D Crea S vacía

> Primer modelo: el primer elemento está disponible nada más comenzar (DP).

110. ¿Cuáles son las primitivas de creación del primer modelo?

A Iniciar, EsVacía y EsÚltimo

B Leer y Escribir

C Crear (S), Registrar (S, e) y Marcar (S) ✔

D Comenzar, Avanzar y EA

Comenzar, Avanzar y EA son las de consulta. Iniciar, EsVacía y EsÚltimo son del segundo modelo.

111. La parte izquierda de la secuencia (Piz) es…

A El elemento actual

B Los elementos que faltan por tratar

C La marca de fin

D Los elementos ya recorridos y tratados (los que quedan a la izquierda del elemento actual) ✔

La cinta se divide en parte izquierda, elemento actual (EA, en la ventana) y parte derecha.

112. El invariante de una iteración es…

A Una variable que no cambia nunca

B La condición de parada

C Una propiedad que se cumple en todos los estados intermedios, en cada paso de la iteración ✔

D La primera instrucción del bucle

Los estados intermedios no son iguales, pero todos comparten esa propiedad; caracteriza el diseño iterativo.

113. Invariante de «longitud de un texto»:

A conta = 0

B conta = número de elementos de la parte izquierda de S ✔

C EA = MarcaFin

D conta = longitud de S

Al terminar, EA = MarcaFin y Piz = S, así que conta = long (S): invariante + condición de parada ⇒ postcondición.

114. ¿Cómo se obtiene la inicialización de cada variable en el análisis iterativo?

A Copiando la postcondición

B Leyendo el primer elemento

C Preguntando cuál es la solución para la secuencia vacía ✔

D Poniendo siempre 0

Son los casos base de la inducción: suma ← 0, numElem ← 0…

115. ¿Cómo se obtiene el tratamiento elemental del cuerpo?

A Suponiendo conocida la solución para Piz (hipótesis de inducción) y calculando la de Piz más el elemento actual ✔

B Con la solución de la secuencia vacía

C Tratando sólo el último elemento

D Probando valores hasta que funcione

v ← g (v, EA): por ejemplo suma ← suma + EA (S).

116. En los esquemas de recorrido, H1 y H2 significan…

A H1: la secuencia vacía se puede tratar como las demás; H2: el primer elemento se puede tratar como los demás ✔

B H1: primer modelo; H2: segundo modelo

C H1: recorrido; H2: búsqueda

D H1: hay marca de fin; H2: no la hay

Según se cumplan o no salen los esquemas 1 (H1 y H2), 2 (¬H1 y H2) y 3 (¬H1 y ¬H2).

117. El esquema 3 de recorrido (¬H1 y ¬H2) usa…

A Sólo un MIENTRAS

B Un RECORRIENDO

C SEGÚN para la secuencia vacía y un ITERAR con el primer elemento tratado aparte ✔

D SEGÚN y un REPETIR

Tratamiento del primer elemento; ITERAR Avanzar; DETENER EA = MarcaFin; tratamiento de EA FIN_ITERAR.

118. Para calcular el máximo de una secuencia de enteros se usa el esquema 3 porque…

A Siempre hay que usar ITERAR con enteros

B Es una búsqueda

C Hay que tratar la marca de fin

D El máximo no está definido para la secuencia vacía, y el primer elemento es el valor inicial ✔

Inicialización: m ← EA (S) (solución para una secuencia de un elemento); cuerpo: SI EA (S) > m ENTONCES m ← EA (S).

119. ¿Cuál es la longitud de la meseta mayor de [35, 18, 18, 18, 5, 62, 35, 35, 11, 11, 11, 11, 9, 9]?

A 2

B 4 ✔

C 14

D 3

La meseta de 11 tiene 4 elementos; la de 18, 3.

120. En la meseta mayor con el esquema 1, el «truco sucio» consiste en…

- A Tratar la marca de fin como un elemento más
- B No inicializar m
- C Saltarse el primer elemento
- **D Inicializar u (el último) con un valor ≤ 0 que no puede aparecer en una secuencia de enteros positivos ✔**

> u no está definido para la secuencia vacía (¬H2); como los datos son positivos, u ← 0 hace que el primer elemento empiece una meseta nueva.

121. En S = [34, 1, 90, 39, 1, 15, 90, 25, 1, 10], ¿cuánto valen pma, uma y nma (primera posición, última y nº de apariciones del máximo)?

- A 3, 3 y 1
- B 7, 3 y 2
- **C 3, 7 y 2 ✔**
- D 2, 9 y 3

> El máximo 90 aparece en las posiciones 3 y 7. (Para el mínimo 1: pmi = 2, umi = 9, nmi = 3.)

122. En máximos y mínimos, la última posición del máximo (uma) se actualiza cuando…

- A EA (S) > ma
- **B EA (S) ≥ ma ✔**
- C EA (S) < ma
- D EA (S) = ma únicamente

> Si EA > ma hay nuevo máximo (y su última posición es la actual); si EA = ma, es otra aparición más tardía. pma sólo cambia con >.

123. Intercalar las secuencias decrecientes [25, 11, 3] y [38, 14, 11] da…

- A [38, 25, 14, 11, 3]
- B [3, 11, 11, 14, 25, 38]
- **C [38, 25, 14, 11, 11, 3] ✔**
- D [25, 11, 3, 38, 14, 11]

> En cada paso se registra el mayor de los dos elementos actuales (o los dos si son iguales) y al final se copia el resto.

124. La postcondición de una búsqueda del primer elemento que cumple Pro es…

- A Todos los elementos cumplen Pro
- **B (Pro (EA) o EA = MarcaFin) y ningún elemento de Piz cumple Pro ✔**
- C EA = MarcaFin
- D Pro (EA) siempre

> El invariante es «ningún elemento de Piz cumple Pro».

125. La condición de continuación del esquema de búsqueda es…

A `(EA = MarcaFin) O Pro (EA)`

B `(EA ≠ MarcaFin) Y NO Pro (EA)` ✔

C `Pro (EA)`

D `EA ≠ MarcaFin`

> Se sigue si no se ha llegado al final y el elemento actual no cumple la propiedad.

126. `YDESPUÉS` (Y condicional) se usa para…

A **Evaluar la segunda condición sólo si la primera es verdadera, y no acceder a EA cuando ya no está disponible** ✔

B Evaluar las dos condiciones en cualquier orden

C Repetir la búsqueda después

D Ejecutar dos acciones seguidas

> Necesario cuando la marca de fin no es significativa o no existe (FDS): `MIENTRAS NO FDS YDESPUÉS NO Pro (EA)`. En C++ `&&` ya se evalúa así.

127. Una búsqueda con garantía de éxito…

A No necesita invariante

B **Puede omitir la comparación con la marca de fin, porque se sabe que algún elemento cumple la propiedad** ✔

C No necesita Avanzar

D Siempre encuentra el primer elemento

> Ejemplo: saltar blancos comparando la marca con ' ' (la comparación da Falso y para).

128. ¿Por qué las secuencias se pasan como parámetros dato–resultado aunque no se modifiquen sus valores?

A Porque son muy grandes

B Porque así lo exige el tipo Carácter

C Porque las acciones no admiten parámetros dato

D **Porque avanzar modifica su estado interno (el elemento actual)** ✔

> Es la justificación del boletín para BuscarLetra (dato c; dato–resultado S).

129. En «¿contiene el texto 'A', 'E' e 'I' en este orden?» con tres llamadas seguidas a BuscarLetra sin comprobar nada entre ellas…

A **El resultado sigue siendo correcto: si una búsqueda llega a la marca, las siguientes no avanzan y se responde «No»** ✔

B Falla siempre

C Entra en un bucle infinito

D Sólo falla si no hay 'A'

> BuscarLetra en la marca de fin no hace nada (la condición del MIENTRAS es falsa). Pero no serviría para 'A', 'A', 'E': tras encontrar la primera 'A', la segunda búsqueda la encuentra otra vez sin avanzar.

130. Para saber si dos secuencias son iguales se…

A Recorre sólo la primera

B Comparan sus longitudes

C Suman sus elementos

D Busca su primera diferencia recorriéndolas a la vez; son iguales si las dos llegan a la marca de fin ✔

> MIENTRAS (EA (S1) ≠ MarcaFin) Y (EA (S2) ≠ MarcaFin) Y (EA (S1) = EA (S2)) HACER Avanzar ambas.

131. «Qué elementos de S1 están en S2» con S1 y S2 ordenadas decrecientemente se resuelve con…

A Una búsqueda en S1 y luego un recorrido de S2

B Un recorrido de S2 que recomienza S1 cada vez

C Un recorrido de S1 con una búsqueda en S2 anidada que continúa donde quedó la anterior ✔

D Dos recorridos independientes

> Como están ordenadas, no hace falta recomenzar S2 para cada elemento de S1.

132. «¿Hay en S2 algún elemento igual a la suma de S1?» combina…

A Un recorrido de S1 seguido (en secuencia) de una búsqueda en S2 ✔

B Una búsqueda en S1 anidada en un recorrido de S2

C Dos búsquedas anidadas

D Un único recorrido

> Primero se suma S1 (¬H1: la suma de la vacía no está definida) y luego se busca ese valor en S2.

133. ¿Qué combinación de características define el primer modelo de acceso secuencial?

A ¬DP y ¬FS

B DP y FS: primer elemento disponible tras comenzar y final detectado al sobrepasar el último ✔

C DP y ¬FS

D ¬DP y FS

> Segundo modelo: ¬DP y ¬FS; tercero: DP y ¬FS; cuarto: ¬DP y FS.

134. ¿Qué primitivas tiene el segundo modelo de acceso?

A Comenzar, Avanzar, EA y MarcaFin

B Iniciar, Avanzar, EA, EsVacía y EsÚltimo ✔

C Leer y Escribir

D Crear, Registrar y Marcar

> Tras Iniciar el primer elemento no está disponible (hay que Avanzar) y el final se detecta estando sobre el último (EsÚltimo).

135. En el segundo modelo, el esquema 1 de recorrido es…

A `Iniciar; MIENTRAS NO EsVacía HACER tratar EA FIN_MIENTRAS`

B `Iniciar; REPETIR tratar EA HASTA EsÚltimo`

C `Iniciar; inicialización; MIENTRAS NO EsÚltimo HACER Avanzar; tratar EA FIN_MIENTRAS` ✔

D `Comenzar; MIENTRAS EA ≠ MarcaFin HACER tratar EA; Avanzar FIN_MIENTRAS`

> Como ¬DP, primero se avanza y luego se trata: cambia el orden de los tratamientos.

136. Un esquema algorítmico es…

A Un algoritmo concreto ya resuelto

B Una plantilla de algoritmo aplicable a una clase de problemas, no a uno solo ✔

C Una secuencia de datos

D Un diagrama de flujo

> Instanciar un esquema es particularizar sus acciones (inicialización, tratamiento, terminación) al caso concreto.

137. Al copiar S1 en S2 con un recorrido, el invariante es…

A S1 = S2

B long (S2) = 0

C Piz (S1) = Piz (S2) ✔

D EA (S1) = MarcaFin

> Comenzar (S1); Crear (S2); MIENTRAS EA (S1) ≠ MarcaFin HACER Registrar (S2, EA (S1)); Avanzar (S1) FIN_MIENTRAS; Marcar (S2).

C++ · C++ de prácticas

138. En Geany, ¿qué teclas compilan (construyen) y ejecutan el programa?

A Ctrl+S construye y Ctrl+R ejecuta

B F1 y F2

C F5 construye y F9 ejecuta

D F9 construye y F5 ejecuta ✔

> También desde el menú Construir o con los botones de acceso rápido.

139. ¿Qué precauciones se piden al guardar un programa?

A Extensión .txt y cualquier nombre

B Guardarlo siempre en el escritorio

C Extensión .cpp, sin espacios ni caracteres no anglosajones en el nombre ni en la ruta, y un nombre significativo ✔

D No hace falta guardarlo antes de compilar

> Geany reconoce el tipo de archivo por la extensión (y entonces resalta la sintaxis).

140. ¿Qué devuelve `main` y qué significa un 0?

A Un booleano; 0 indica error

B Nada (void)

C Un entero; 0 indica al sistema operativo una finalización normal ✔

D Un real con el tiempo de ejecución

> Si no aparece return, el sistema recibe 0 por omisión.

141. En C++, `notaFinal` y `notafinal`…

A Son identificadores distintos (C++ distingue mayúsculas y minúsculas) ✔

B Son el mismo identificador

C Son palabras reservadas

D Dan error por ser parecidos

> Por eso en el programa con errores de la práctica 4 `Dato1` y `dato1` no son la misma variable.

142. ¿Para qué sirve `using namespace std;`?

A Para declarar main

B Para activar los comentarios

C Para incluir iostream

D Para usar directamente cin, cout y el resto de la biblioteca estándar, que están en el espacio de nombres std ✔

> Sin ella habría que escribir std::cout, std::cin…

143. ¿Cómo se escriben en C++ los operadores de la notación DIV, MOD, ≠, = (comparación), Y, O, NO?

A `//` , `%` , `≠` , `=` , `&&` , `||` , `!`

B `/` , `%` , `≠` , `==` , `&&` , `||` , `!` ✔

C `/` , `%` , `<>` , `=` , `&` , `|` , `~`

D `div` , `mod` , `<>` , `=` , `and` , `or` , `not`

> Ojo: `=` en C++ es la asignación (←); la comparación es `==`.

144. En C++, con `int`: `7 / 2`, `7 % 2` y `float(7) / 2` valen…

A **3, 1 y 3.5 ✔**

B 4, 1 y 3.5

C 3, 1 y 3

D 3.5, 1 y 3.5

`/` entre dos int es la división entera (DIV). Basta convertir uno de los operandos a real para obtener la división real.

145. En C++, `-7 / 2` y `-7 % 2` valen…

A −4 y 1

B −3 y 1

C **−3 y −1 ✔**

D −4 y −1

La división entera trunca hacia cero y el resto lleva el signo del dividendo. Por eso para saber si x es impar se compara `x % 2 ≠ 0`, no `x % 2 == 1`.

146. Con `int i; bool text;`, tras `i = 3.6; text = 'A';` se escribe `cout << i << text`. ¿Qué sale?

A 3.6A

B **31 ✔**

C 4true

D 3A

Conversiones implícitas: 3.6 se trunca a 3 y 'A' (distinto de 0) se convierte en true, que cout muestra como 1. Hay que evitarlas.

147. El anexo de la práctica 2 declara `const int PI = 3.14159;`. ¿Cuánto vale PI?

A 3.14159

B 3.14

C Da error de compilación

D **3 ✔**

Al ser int se trunca. Debería ser `const float PI = 3.14159;` (o double).

148. ¿Cuál es el rango de un `int` de 4 bytes?

A **−2147483648 … 2147483647 ✔**

B 0 … 4294967295

C −32768 … 32767

D −128 … 127

char: −128…127 (1 byte); short: −32768…32767 (2 bytes).

149. ¿Cuál es el mayor n cuyo factorial cabe en un `int` de 4 bytes?

- A 13
- B 31
- **C 12 ✔**
- D 20

> 12! = 479 001 600 < 2 147 483 647, pero 13! = 6 227 020 800 se desborda. Con `long long` se llega a 20!.

150. ¿Por qué no conviene comparar reales con `==`?

- **A Porque los cálculos con reales son aproximados; mejor comprobar que |a − b| < EPS ✔**
- B Porque `==` asigna
- C Porque `==` sólo vale para caracteres
- D Porque los reales no se pueden comparar

> Por ejemplo, con float `1.0 - 0.8 - 0.2` no da exactamente 0 (da $\approx -1.5 \cdot 10^{-8}$), mientras que `1.0 - 0.2 - 0.8` sí.

151. Con el buffer `1234␣␣␣␣` y `char c; int i; cin >> c; cin >> i;`, ¿qué valores quedan?

- **A c = '1' e i = 234 ✔**
- B c = '4' e i = 123
- C c = ' ' e i = 1234
- D c = '1' e i = 1234

> `>>` con char lee un único carácter (no blanco); el entero se lee con lo que queda. Los cuatro espacios siguen en el buffer.

152. Con el buffer `La Luna del Valle.` y tres `cin >> c` (char c), c toma sucesivamente…

- A 'L', 'u', 'n'
- B 'L', 'a', ' '
- C "La", "Luna", "del"
- **D 'L', 'a', 'L' ✔**

> `>>` se salta los separadores (blancos, tabuladores, saltos de línea). Para leerlos se usa `cin.get (c)`.

153. ¿Cómo se lee una línea completa en un string?

- A `cin.get (cadena)`
- **B `getline (cin, cadena)` ✔**
- C `fflush (cadena)`
- D `cin >> cadena`

> `cin >> cadena` lee una sola palabra (hasta el primer blanco). El primer programa de la práctica 1 usa getline para leer el nombre completo.

154. ¿Qué diferencia hay entre `cin >> c` y `cin.get (c)` con un char?

- A `>>` lee una línea entera
- B `get` sólo lee dígitos
- **C `>>` se salta blancos y saltos de línea; `get` lee cualquier carácter, incluidos ' ' y '\n' ✔**
- D Ninguna

> Por eso para contar blancos o leer hasta el fin de línea ('\n') se usa cin.get.

155. Restricciones del `switch` de C++ frente al SEGÚN:

- A Admite cualquier condición booleana en cada caso
- B No admite default
- C Sólo funciona con reales
- **D Una única expresión de tipo ordinal comparada con valores estáticos (literales o constantes) ✔**

> Por eso el SEGÚN general se traduce con `if … else if …` y el switch sólo cuando se compara una expresión ordinal con constantes.

156. Si en un `switch` un `case` no termina en `break`…

- A Se vuelve a evaluar la expresión
- B Da error de compilación
- C Se ejecuta el default
- **D Se siguen ejecutando las sentencias de los case siguientes hasta encontrar un break ✔**

> Esa es la forma de asociar varios valores a un mismo tratamiento: `case 19: case 18: … break;`.

157. ¿Cómo se declara un parámetro dato–resultado real llamado a?

- A `ref float a`
- **B `float &a` ✔**
- C `float a`
- D `&float a`

> Sin & el paso es por valor (parámetro dato).

158. Una función en C++…

- A Devuelve los resultados por parámetros &
- B No puede devolver un struct
- C Se declara con void
- **D Se declara con el tipo del resultado en lugar de void y devuelve el valor con return; sus parámetros son por valor ✔**

> C++ permite devolver tanto escalares como tipos estructurados (un Punto, una Fecha…).

159. Con `struct Punto { float abscisa, ordenada; }; struct Cuadrado { Punto p1, p2, p3, p4; };` y `Cuadrado c;`, la abscisa del tercer vértice es…

A `c.p3.abscisa` ✔

B `c[p3].abscisa`

C `p3.c.abscisa`

D `c.abscisa.p3`

El operador punto se encadena: primero el campo p3 de c y luego su campo abscisa.

160. En la práctica 4 (perímetro), `struct Punto` está definido dentro de main. ¿Qué problema da al definir `float Distancia (Punto p, Punto q)`?

A Que Distancia no puede devolver float

B Que los struct no pueden ser parámetros

C Ninguno

D Punto no es visible fuera de main: hay que definir el tipo en el léxico global, antes de las funciones ✔

Los tipos que usan varios subprogramas van en el léxico global del programa.

161. El programa de la práctica 4 con BinDec e Intercambio (x='O', y='P', z='Q'; Intercambio (x, y); Intercambio (y, z); BinDec (b1, b2, b1, b2, …) con b1 = 0, b2 = 1; BinDec (0, 1, b1, b1, …)) escribe…

A PQO, 5 y 1

B QOP, 10 y 4

C PQO, 10 y 2 ✔

D OPQ, 5 y 2

(O,P,Q) → (P,O,Q) → (P,Q,O). dec = b1 + 2·b2 + 4·b3 + 8·b4: 0 + 2 + 0 + 8 = 10 y 0 + 2 + 0 + 0 = 2.

162. ¿Cómo se traduce `REPETIR S HASTA cond` a C++?

A `do { S } while (cond);`

B `repeat { S } until (cond);`

C `do { S } while (!(cond));` ✔

D `while (cond) { S }`

La condición de C++ es de continuación: hay que negar la de parada.

163. `i RECORRIENDO [a, b] EN SENTIDO INVERSO HACER S` en C++ es…

A `for (i = b; i > a; i = i - 1) { S }`

B `for (i = b; i ⩾ a; i = i - 1) { S }` ✔

C `while (i ⩾ a) { S }`

D `for (i = a; i ⩽ b; i = i + 1) { S }`

El recorrido normal es `for (i = a; i ⩽ b; i = i + 1)`.

164. Según el boletín de la práctica 5, ¿cuándo se puede usar `break`?

A Nunca

B Sólo para la parada central de un ITERAR en un `do { … } while (true);` ✔

C Para salir de cualquier bucle cuando se quiera

D Sólo dentro de un for

> Usarlo para salir de otro tipo de iteración se considera un grave fallo de concepto (además de su uso normal en el switch).

165. ¿Cuántos términos de la serie armónica 1 + 1/2 + 1/3 + … hacen falta para que la suma supere 5?

A 83 ✔

B 150

C 5

D 25

> Es el ejemplo 2 de la práctica 5. La serie crece muy despacio (como ln n).

166. En el primer modelo con la entrada estándar, ¿dónde van las lecturas?

A Sólo una después del bucle

B Una lectura antes del bucle (Comenzar) y otra al final del cuerpo (Avanzar) ✔

C Una en la condición del while

D Sólo una al principio del cuerpo

> `cin >> EA; while (EA =≠ MF) { tratar EA; cin >> EA; }`.

167. En la práctica 6, la versión que hace `cont = 0; while (EA =≠ MF) { cin >> EA; … }` sin leer antes del bucle es incorrecta porque…

A Lee dos veces cada dato

B Compara EA sin haberle dado valor y trata la marca de fin como un dato más ✔

C No compila por usar while

D Cuenta un elemento de menos

> La primera comparación usa basura y, cuando se lee el −1, se escribe y se cuenta antes de salir. (Además, las dos versiones usan EA_S, que no está declarada.)

168. Para leer una línea carácter a carácter hasta el final se usa…

A `cin.get (c); while (c =≠ '\n') { …; cin.get (c); }` ✔

B `cin >> c; while (c =≠ '\n') { …; cin >> c; }`

C `while (cin >> c)`

D `getline (cin, c)` con c char

> Con `>>` nunca llegaría el '\n' (se lo salta) y el bucle no terminaría en el fin de línea.

169. Para buscar el primer número par en una secuencia de enteros que pueden ser negativos, la condición de continuación correcta es…

A `(EA ≠ MF) && (EA % 2 ≠ 0)` ✔

B `(EA ≠ MF) && (EA % 2 == 1)`

C `(EA ≠ MF) || (EA % 2 ≠ 0)`

D `EA % 2 ≠ 0`

> Con negativos `-3 % 2` vale −1, así que `== 1` no reconoce todos los impares. Y sin `EA ≠ MF` se pasaría de la marca.

170. ¿Cuál es la forma de C++ del ITERAR que propone el boletín sin break?

A `S1; while (!(cond)) { S2; S1; }` ✔

B `for (;;) { S1; S2; }`

C `do { S1; S2; } while (cond);`

D `while (cond) { S1; S2; }`

> Es la equivalencia ITERAR S1 DETENER cond S2 ≡ S1; MIENTRAS NO cond HACER S2; S1.